

Accelerating Machine Learning Kernels on Transmuter and GPU

Radhika Dang, Arpit Jaiswal, Shubham Parmar, Naman Saxena
Electrical Engineering & Computer Science, University of Michigan, Ann Arbor, MI

Introduction

Neural networks are everywhere today. From classifiers to pattern recognition and data mining, they perform numerous computations on ever-changing and expanding datasets. K-Nearest Neighbours (KNN) and Graph Convolutional Networks (GCN) are some of the widely used algorithms for classification and modeling graph-structured data, respectively. GCNs are efficient for inherently relational data and allow for the exploration of tremendous applications in communication networks, chemistry, neurology, and other varied fields [1]. While their novelty provides tremendous exploration opportunities, they are difficult to compute due to huge graph sizes, a mix of dense and sparse operations, and inherent dependence on an input graph. Transmuter [2] alleviates these issues by adapting to changes in kernel data reuse and control divergence characteristics as it can reconfigure the on-chip memory type, resource sharing, and dataflow during runtime within a short period. It also supports a rapidly growing set of algorithms exhibiting dynamic data movement patterns, irregularity, and sparsity. It also delivers GPU-like efficiencies for most traditional dense applications, making it an ideal choice for accelerating neural networks.

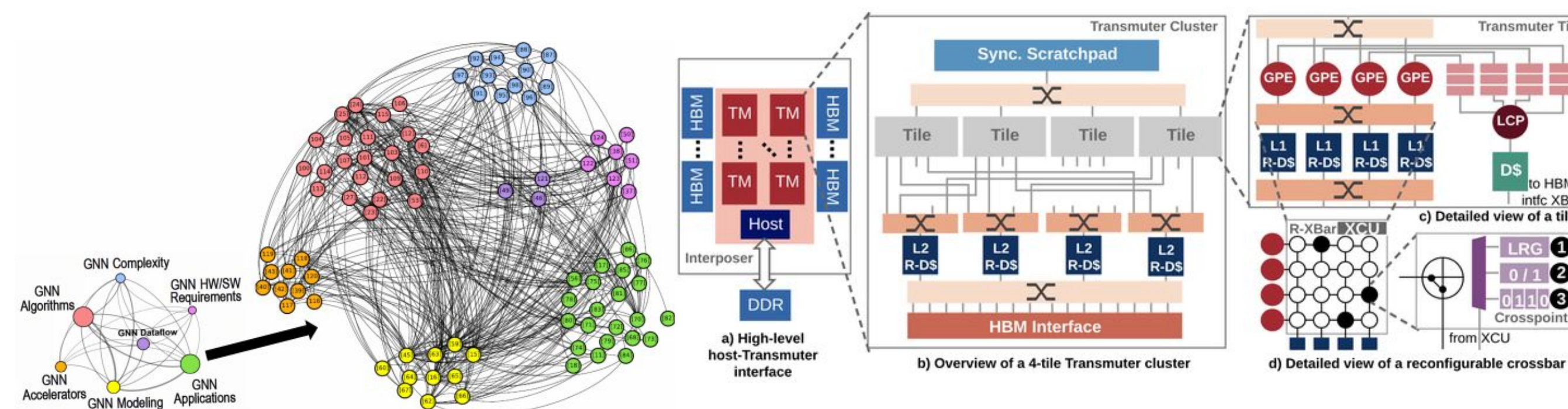


Figure 1: Full knowledge graph representation as of October 2020. [1]

Figure 2: Transmuter architecture. [2]

Objectives and Approach

Given the availability of a state-of-the-art hardware accelerator code base (Transmuter), we focussed on identifying widely utilized algorithms in the current technical scene and accelerating them. We narrowed it down to KNN and GCN, as they are key components in almost all machine learning models. The key objectives of this project focused on implementing these algorithms on baseline C, CUDA, and Transmuter. Further, we conclude our project with thorough profiling of all three versions and identify the advantages and bottlenecks in Transmuter concerning these algorithms.

Our approach toward this project is detailed below:

1. Implement forward & backward pass of GCN model using Pytorch (Google Colab) on CORA and REDDIT dataset and obtain pre-trained weights with validation accuracy of 80%
2. Dataset feature transformations for using with C++ implementation and exploiting CSR optimization in the GCN aggregation phase
3. Understand the Transmuter architecture and codebase
4. Implement all three versions using "IRIS dataset for KNN and CORA dataset for GCN and identify the computational bottlenecks

5. Optimizing CUDA and TM implementations with parallel programming techniques such as Tiled Matrix Multiplication, Compressed Sparse Row Matrix-Matrix multiplication (CSR)
6. Further accelerate the TM version by exploiting hardware knobs and configurations (primarily to run on multiple GPEs and LCPs)
7. Profile C, CUDA, and Transmuter implementations using CHRONO standard library, NVIDIA NSight, and transmuter logs, respectively

Results

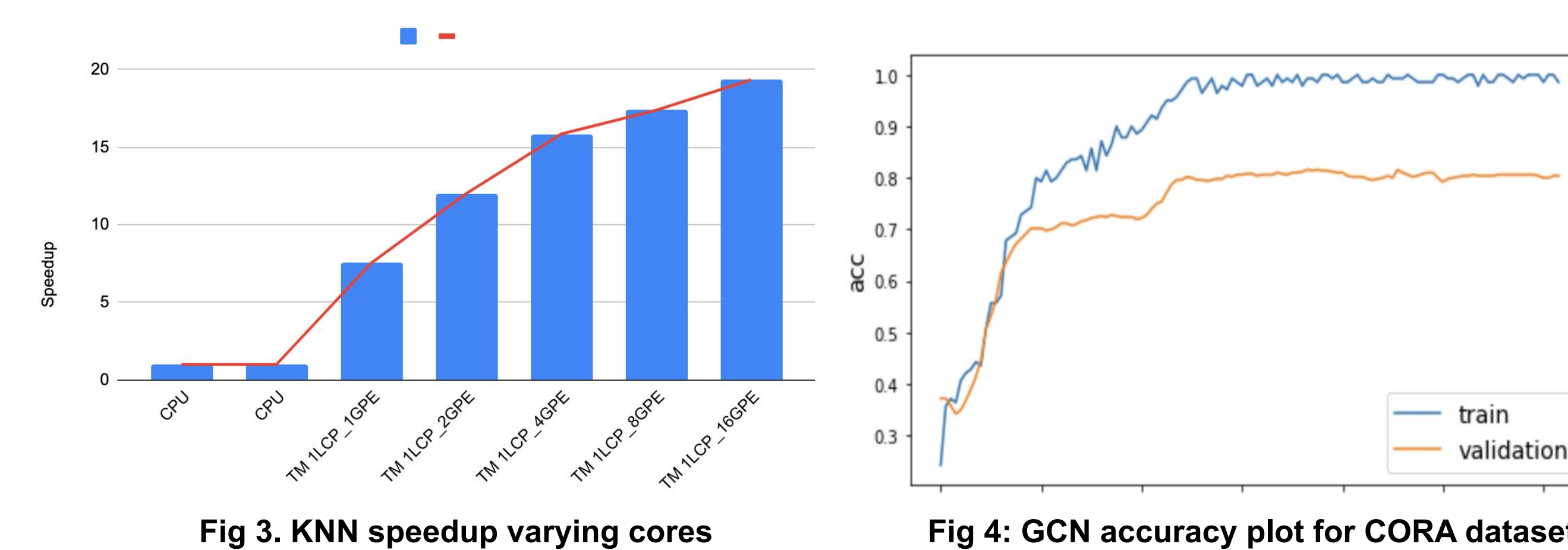


Fig 3. KNN speedup varying cores

Fig 4: GCN accuracy plot for CORA dataset

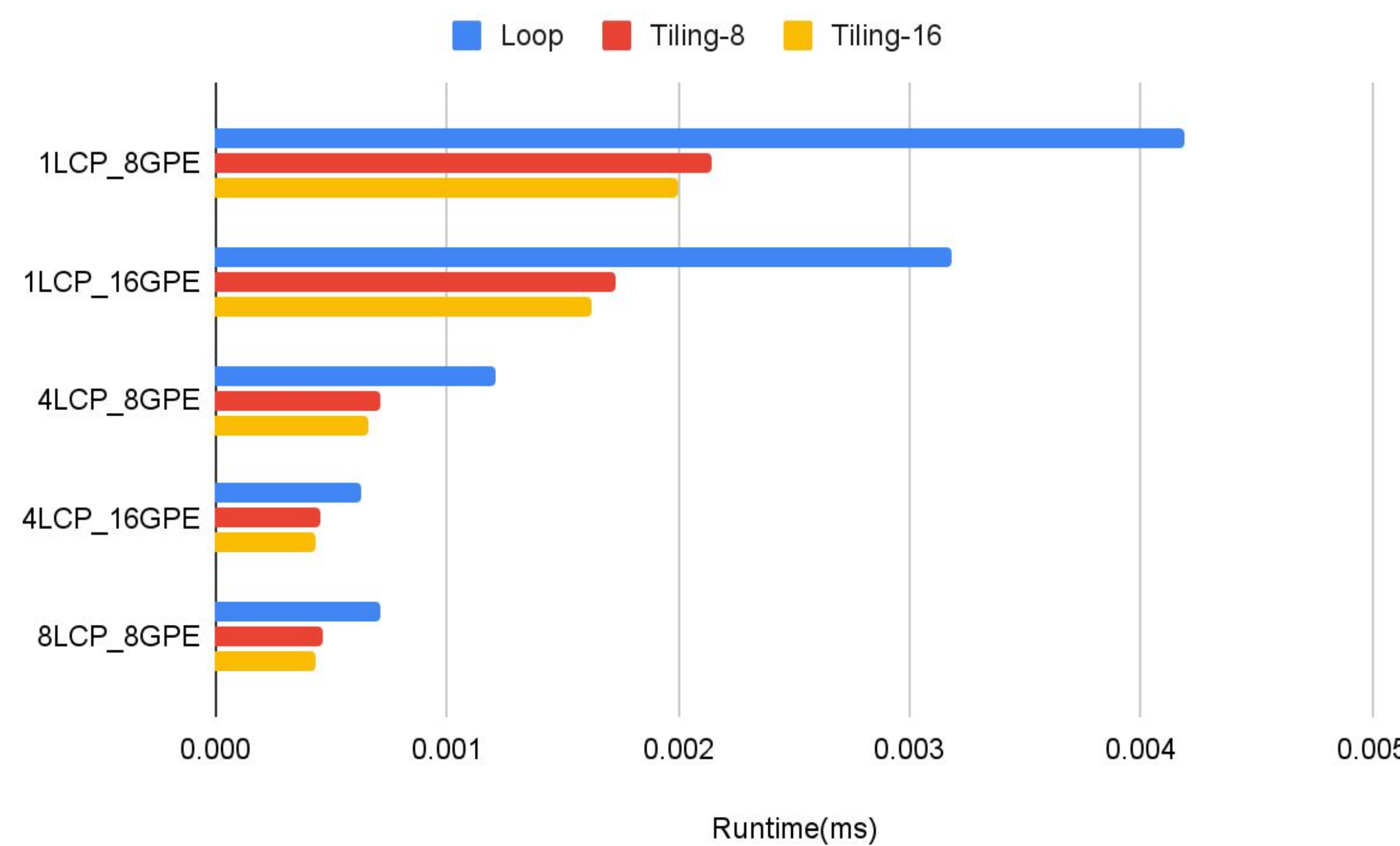


Fig 5. GCN optimization on Transmuter with synthetic 1000 node dataset

Layout	Aggregation		Combination		Memory	Analyzer	Streams	Overall Runtime (5 runs average)	
Transposed	CSR	2D block + one element/thread	Naive	Tiled	Pinned	Reduction	Enabled	Cora	Citeseer
	✓		✓					0.811s	2.923s
✓	✓			✓				0.2248s	0.6682s
✓	✓	✓						0.2048s	0.6100s
✓	✓	✓			✓			0.2016s	0.5950s
✓	✓	✓			✓	✓		0.1930s	0.5878s
	✓	✓			✓	✓	✓	0.1938s	0.5944s

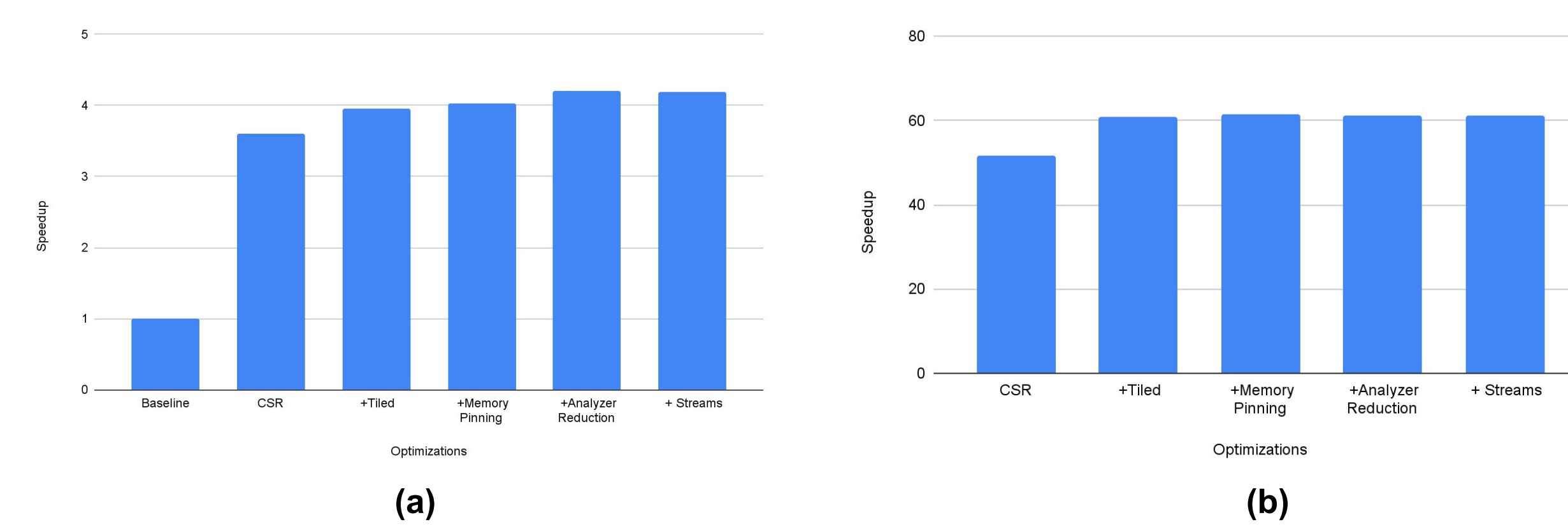


Fig 6. Speedup on CUDA on (a) REDDIT and (b) CORA dataset

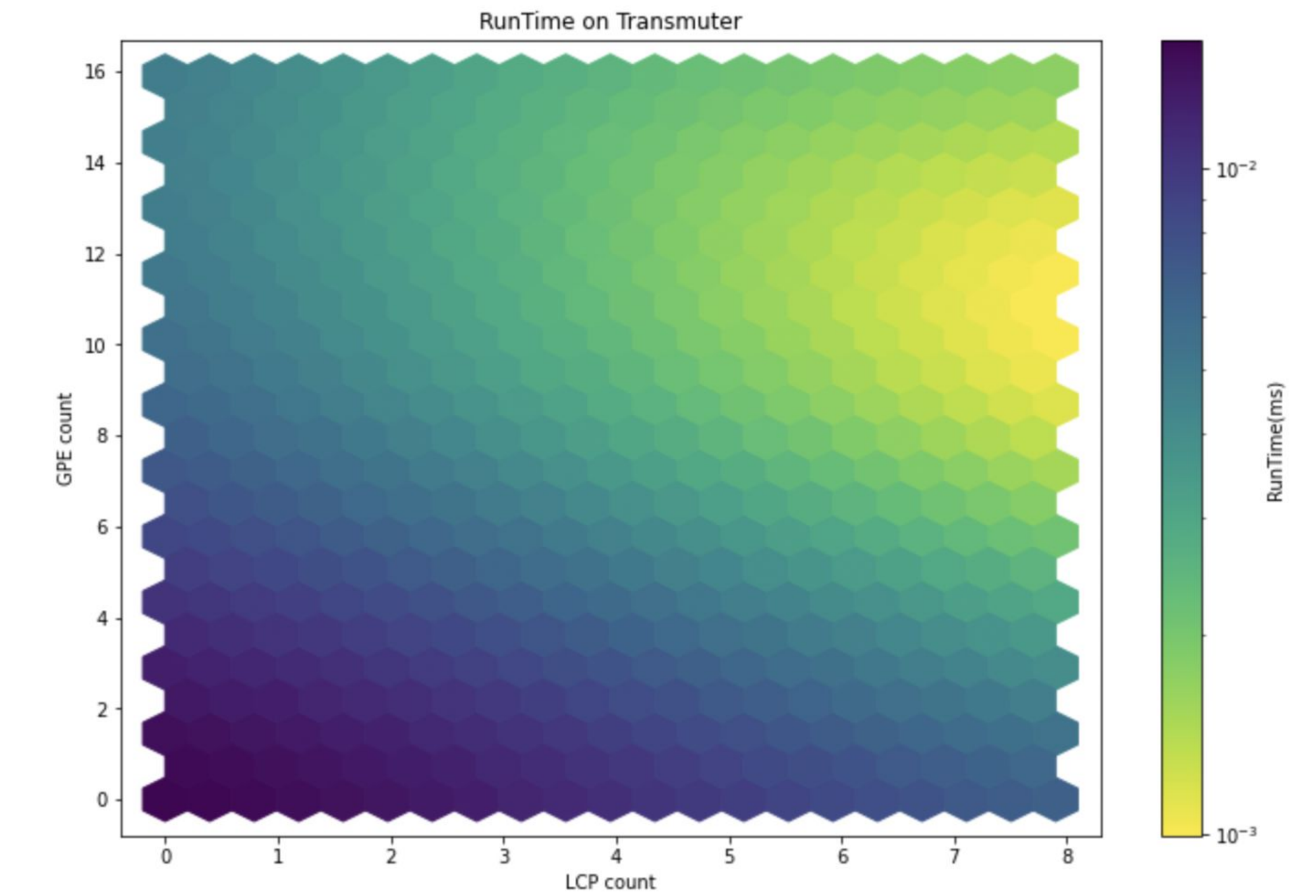


Fig 7. Transmuter run time variation on synthetic dataset of 1000 nodes with varying GPEs and LCPs

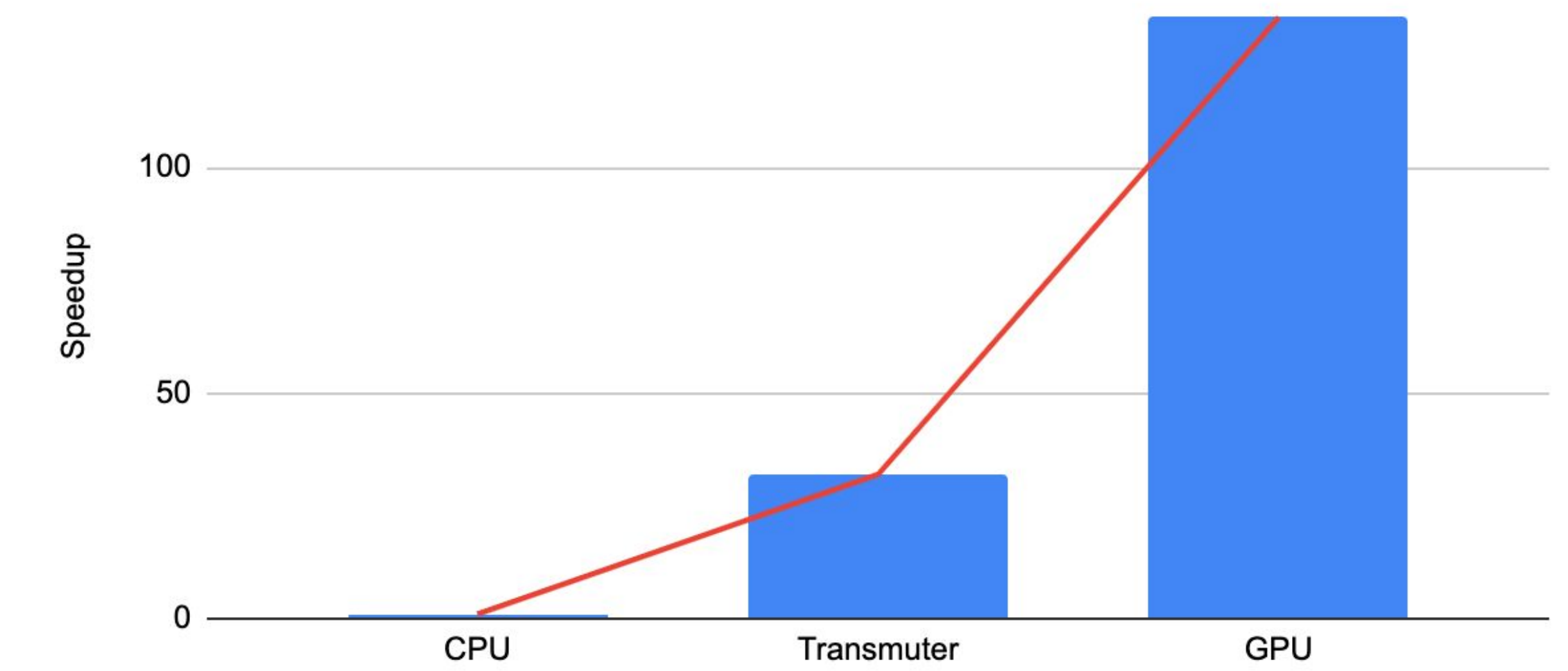


Fig 8. GCN Speed-Up on all three versions

Discussion and Conclusions

We find that sheer brute force GPU and Transmuter are comparable in GCN with the CORA dataset. However, as we move towards larger datasets such as REDDIT, we observe significant performance improvements on GPU. The performance of Transmuter improves with reconfigurable architectures and workloads with varying characteristics. We faced significant challenges in setting up the Transmuter's gem5 environment. There were many programming difficulties because it doesn't support many basic techniques out of the box in the environment.

Acknowledgements

We want to thank Prof. Yatin Manerkar for advising us throughout the project and providing relevant resources. We would also like to thank Subhankar Pal for sharing the Transmuter code base and Siying Feng for technical know-how for using the accelerator.

References

- [1] Abadal et al. (2020). Computing Graph Neural Networks: A Survey from Algorithms to Accelerators.
- [2] Pal et al. (2020). Transmuter: Bridging the Efficiency Gap using Memory and Dataflow Reconfiguration. 175-190. 10.1145/3410463.3414627.
- [3] CORA dataset: <https://relational.fit.cvut.cz/dataset/CORA>